

Interconnects & Network on Chip

NoC NoC!

Who's there?

Bus Terminology

Master/Slave Terminology is outdated, to be avoided where possible!

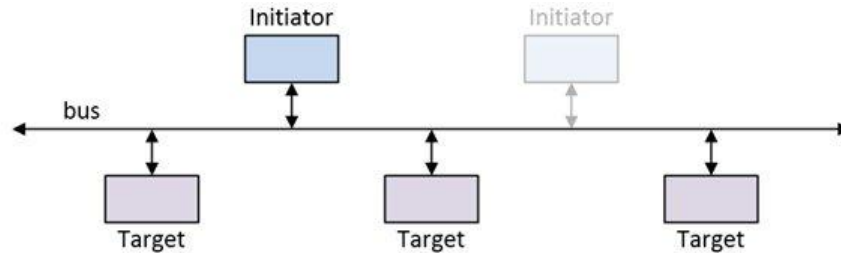
Alternatives Include:

- **Manager / Subordinate** (Used by AXI)
 - Good because it replaces M/S abbreviations for signals like MOSI/MISO
- **Initiator / Target**
 - Makes sense in systems where a device could be both an initiator or a target
- **Controller / Peripheral**
 - Useful Vocab for when interfacing with Memory Mapped IO
 - Controller accesses peripherals

Bus Contention: multiple nodes attempting to access a bus at the same time

Buses

- Simplest Way to connect data sources and data consumers
 - **Initiator:** a Core, an Accelerator, a DMA engine (anything that reads/writes)
 - **Target:** A memory bank, an IO peripheral (anything that gets read from or written to)
 - **Both:** a cache that gets read from but also requests data on miss
- **Bus Contention:** only one node on the bus may be active at any time



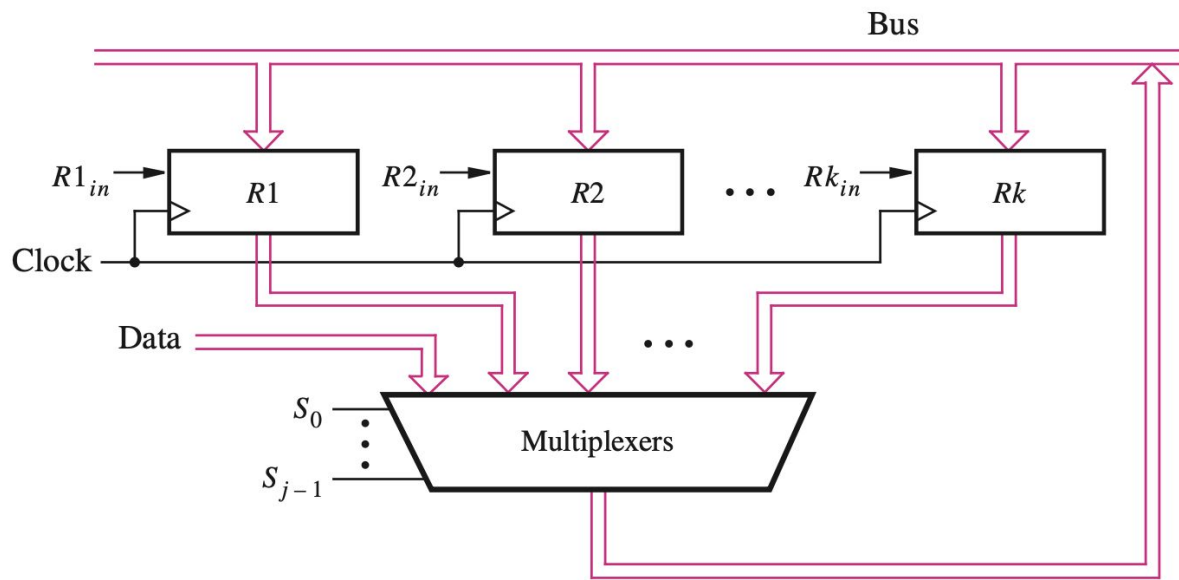
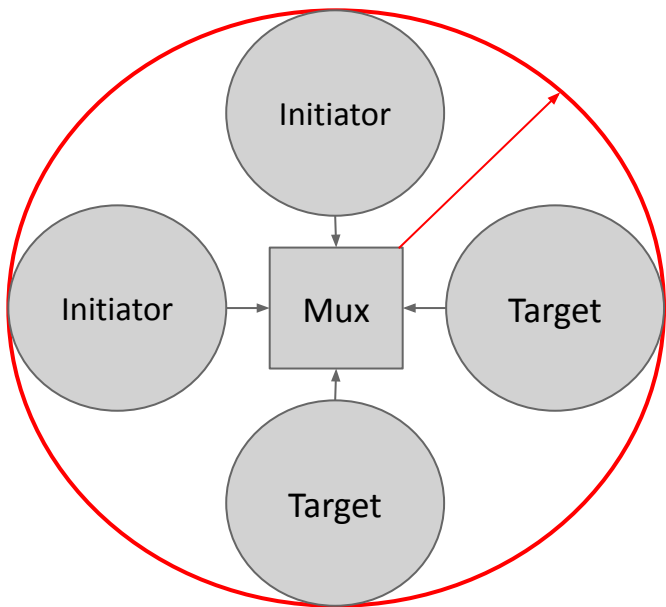
Bus Architecture - Point to Point

- Simplest Form of Interconnect
 - Example: CPU and its stack memory
 - Usually CPU is the only thing writing to and reading from its stack
- **Synchronous Bus**
 - Signals change on clock edges of a shared clock
 - Asynchronous architectures exist, but are very complicated



Accessing the Bus: Shared & Muxed

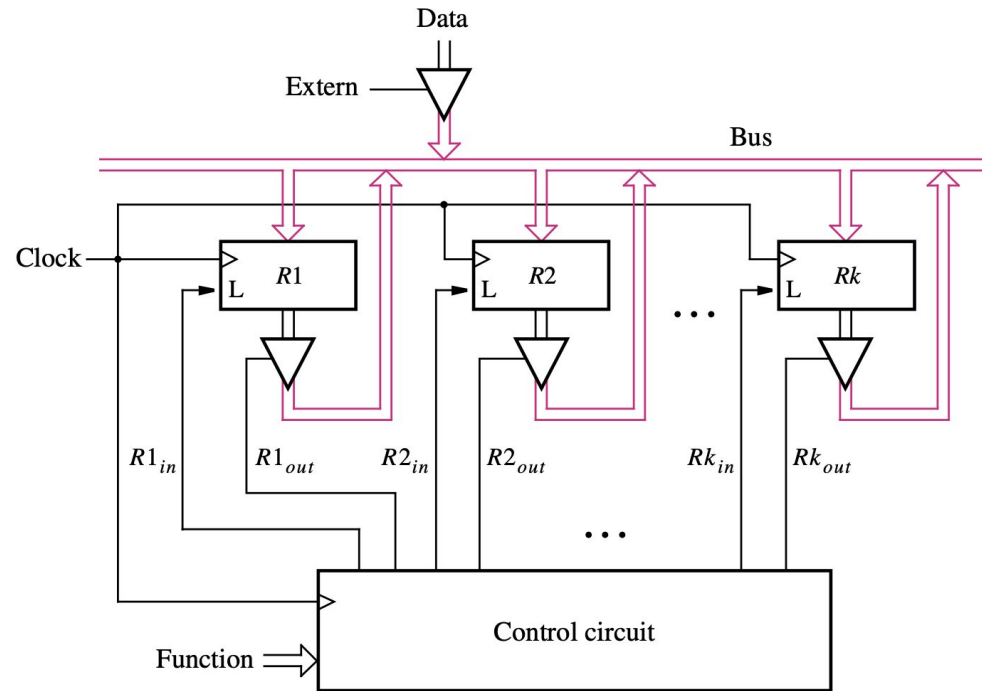
- Bus is controlled by centralized mux and routed to all endpoints
- Signals must be routed from each endpoint to the mux and then back onto the global bus
 - + simple, clear ownership of the bus, error proof
 - Extra wires are expensive, slow, often limiting factor



Accessing the Bus: Shared & Tri-State

- The mux technique was wasteful of wires
 - Route signals both to and from central mux
- What if each endpoint could individually control the bus?
 - Solution: Use tri-state logic
 - Bus is kept High Z, driven by each node
- Problem: What if multiple nodes try to drive the bus?
 - Short circuit would occur
 - Need to control which nodes are allowed to drive the bus
- Tradeoffs:
 - + Many Fewer Wires
 - Large drive circuitry at every node
 - Tri state is complicated and risks shorts

 = tri-state buffer (0, 1, or High Z Out)



Central Bus Arbitration: Daisy Chain

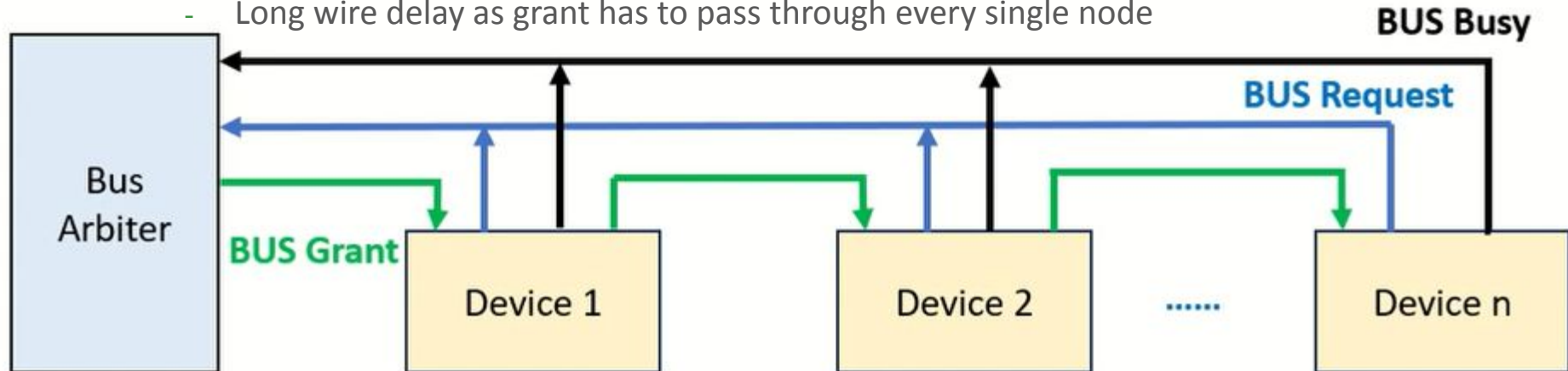
Glossary

How do we control who has access to the bus? → **Bus Arbitration**

Bus Arbitration: control of which devices get to access bus, conflict resolution

Daisy Chain Technique:

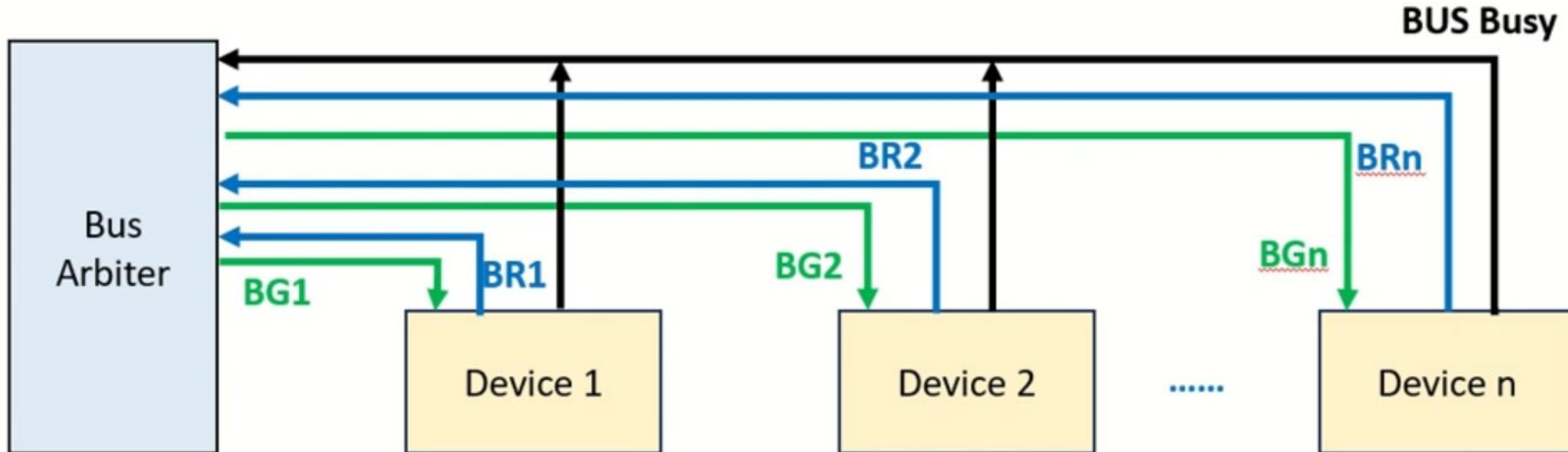
- Each device can drive a global request line to gain access, a busy lane to maintain access
- Grant signal is sent to highest priority device
- Each device can pass the grant signal to the next if it doesn't require access
 - + Simple Logic with minimal wires
 - Unfair access distribution as highest priority always gets first say
 - Long wire delay as grant has to pass through every single node



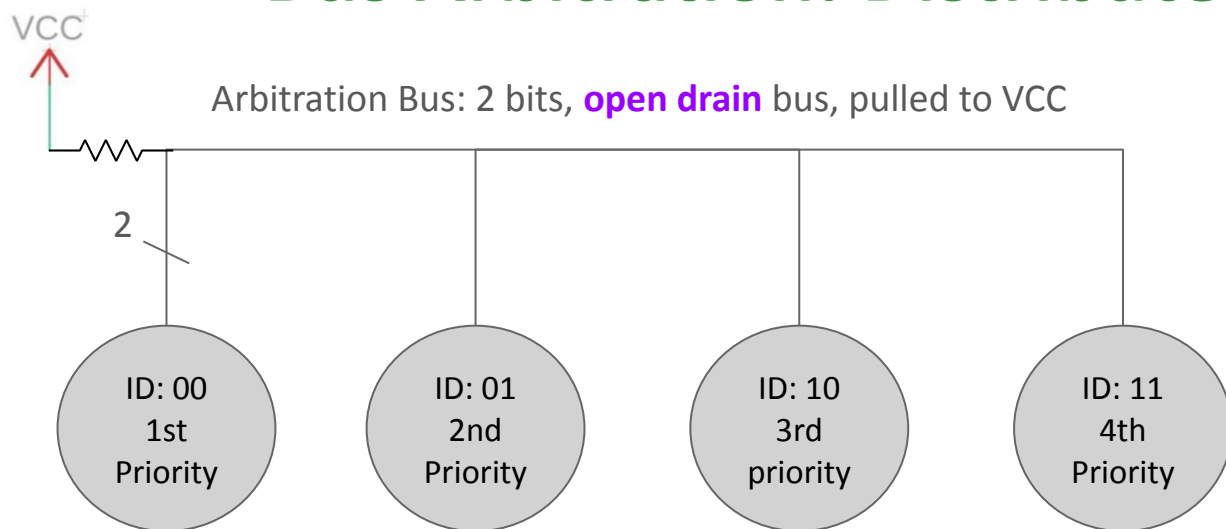
Central Bus Arbitration: Independent

Independent Per-Device Control:

- Each device has its own request, and grant signals (busy can stay shared)
- Bus Arbiter can use algorithm of its choosing to prioritize
 - + Bus arbiter can make intelligent decisions about fairness and access control
 - Uses a lot more wires per device



Bus Arbitration: Distributed



Glossary

1. Node will broadcast its ID,
2. Node checks that its ID is still present on the bus
 - Id of 00 will beat 01
 - ID is not present → another node has pri
3. If so, Node has ownership of bus

Open Drain Logic: wire is pulled high, each node can only drive it low

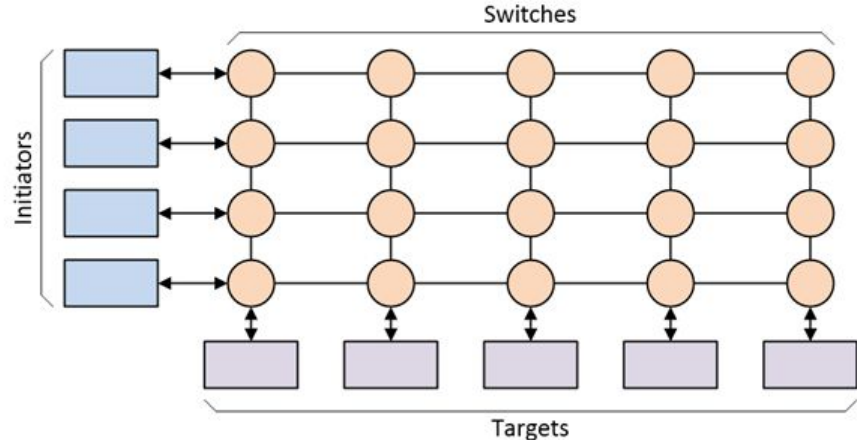
- Allows multiple drivers without causing a short circuit
- If multiple drivers, bus will be pulled to ground
 - 0 will always “beat” a 1

Crossbars

- Every initiator is connected to every target
 - Solves bus contention while initiators are talking to different targets
- Each switch acts as a mux and a queue
 - Mux Initiator's bus to the correct target
 - Enqueue (or reject) new transactions when still busy with a previous transaction

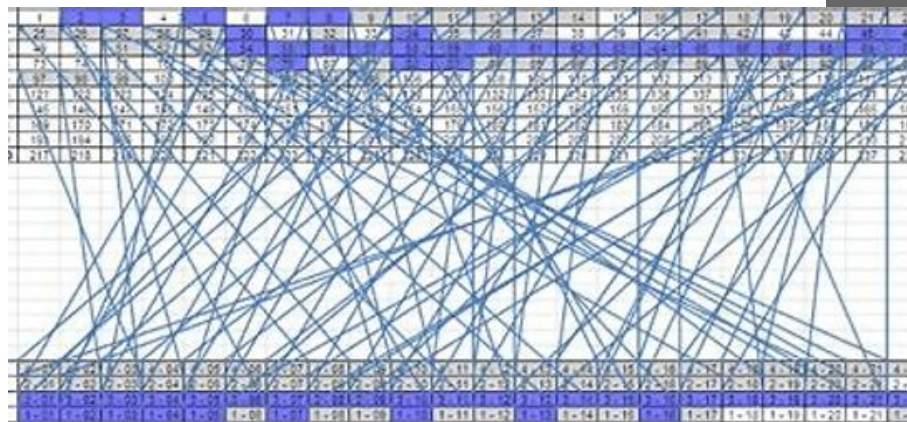


Named after the physical crossbars of hand-operated phone lines



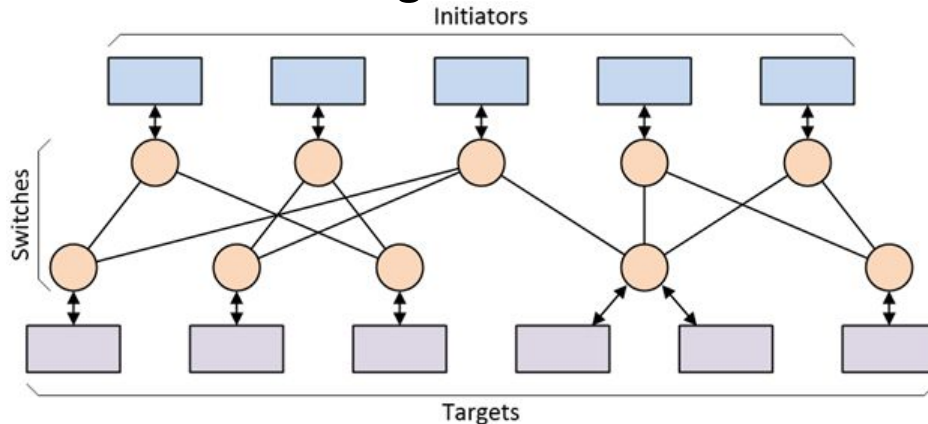
Crossbar Tradeoffs

- + All initiators can access at the same time
- + Better throughput as a whole
- Difficult to lay out and route!
 - 64 parallel wires between each switch and its neighbors
- Quadratic Scaling
 - o Need $(\# \text{ initiators}) * (\# \text{ targets})$ switches
- Works best with fewer nodes



Network On Chip

- Instead of every node being connected to each other, they are connected to network of topology-aware switches
- Bus transactions take the form of packets with headers
 - Allows for fragmenting larger transactions into smaller transactions
- Easier to route: less connections, custom topology to fit application
- Relies on not all nodes being active at the same time



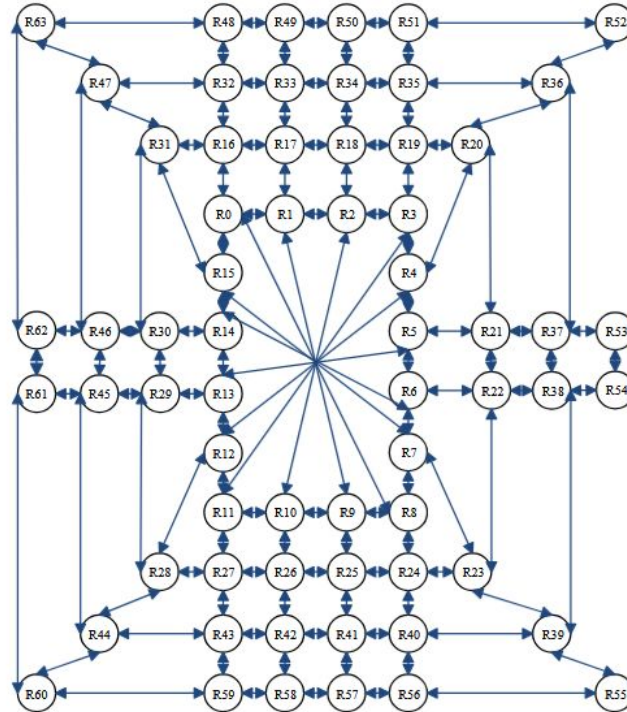


Figure 5. XX-Ring Topology with 64 Nodes

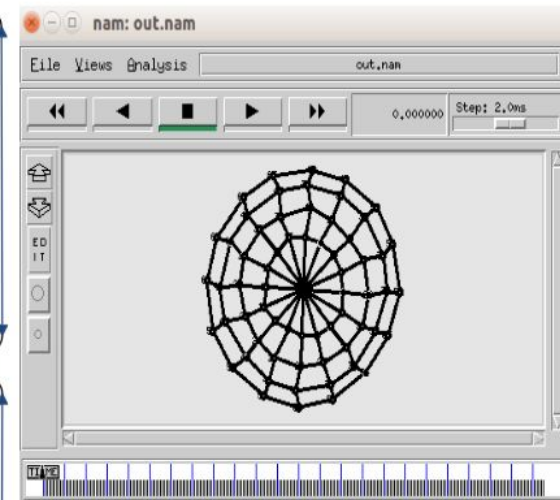
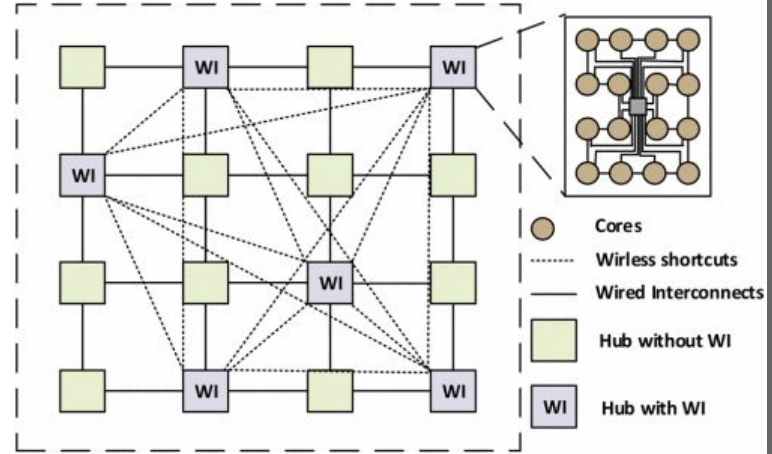


Figure 6. XX-Ring Topology with 64 Nodes
Display in NS-2 Simulator

Wireless Hybrid Network on Chip

- What if there was a way for each node to talk to each other without wires?
 - Oh wait! The universe is literally an electromagnetic field
- Important nodes are equipped with tiny RF antennae & transceivers to directly communicate
- Wire-based NoC still used, but RF added as a faster bypass



Wishbone - Signals

Manager Signals

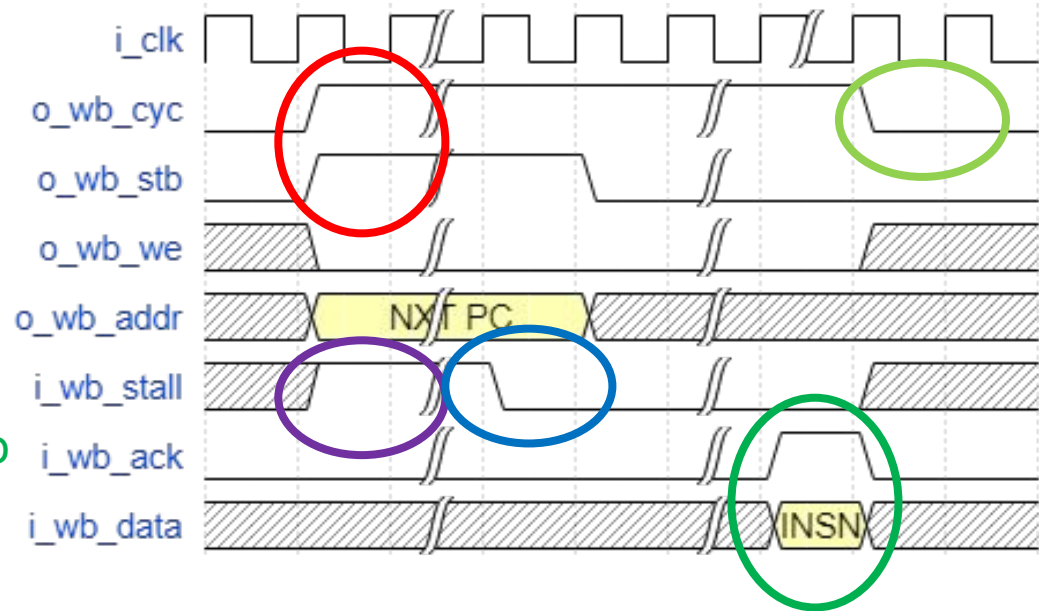
CYC_O	o_wb_cyc	Cycle – Held high during the whole transaction
STB_O	o_wb_stb	Strobe – valid transfer signal, qualifies addr, we, etc.
WE_O	o_wb_we	Write Enable
ADDR_O	o_wb_addr	Address
DATA_O	o_wb_data	Data (Write)
SEL_O	o_wb_sel	Byte Select

Subordinate Signals

STALL_I	i_wb_stall	Stall – not ready to respond yet
ACK_I	i_wb_ack	Acknowledge – transaction is complete
DATA_I	i_wb_data	Data (Read)
ERR_I	i_wb_err	Error

Wishbone – Read Transaction

1. **Cycle** and **Strobe** go high to indicate a new valid read request
2. Target responds with a **stall**, meaning it is not ready! Initiator must continue to hold
3. Subordinate lowers **stall**, so Initiator can stop holding **strobe** and **address**
4. Subordinate raises **Ack** to show valid **data** response
5. **Cycle** only goes low after ack received

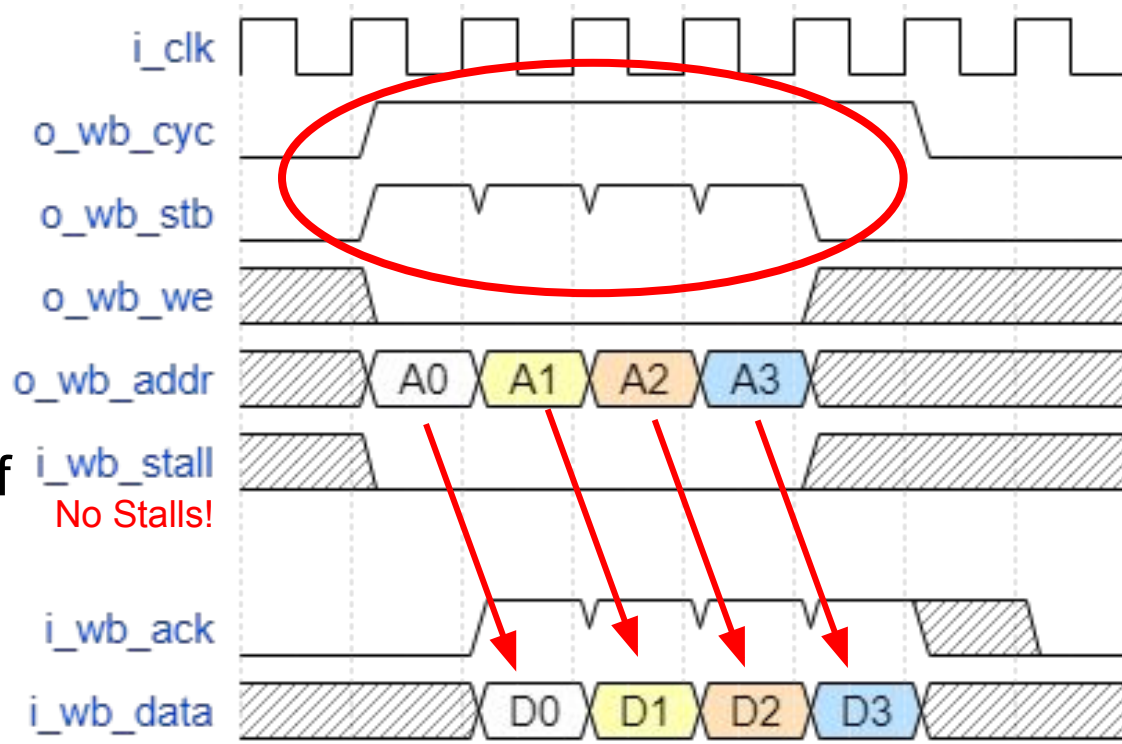


Wishbone - Pipeline

- Repeated **Strobes** without **Stalls** during one **Cycle** means pipelined access!

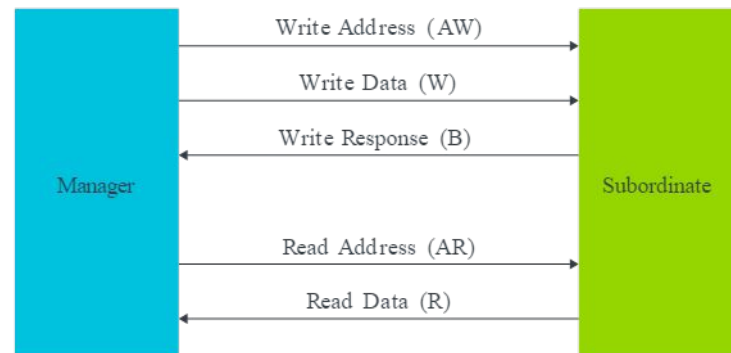
- Would be different if there were stalls

- Allows single cycle memory accesses



AXI

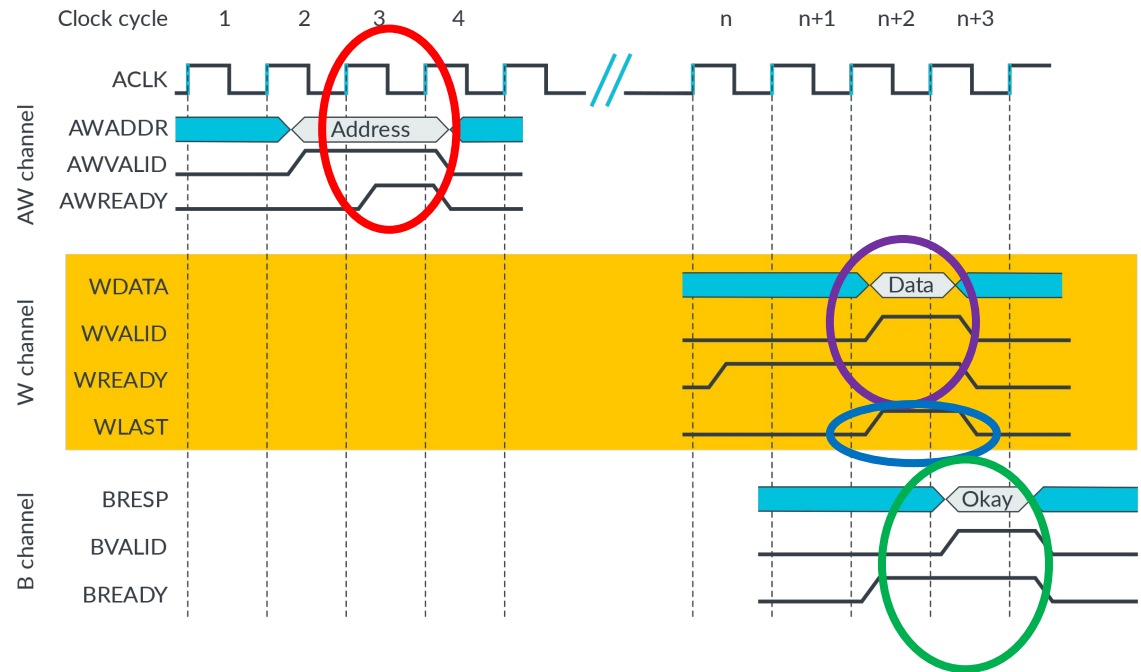
- AXI beat/burst joke...



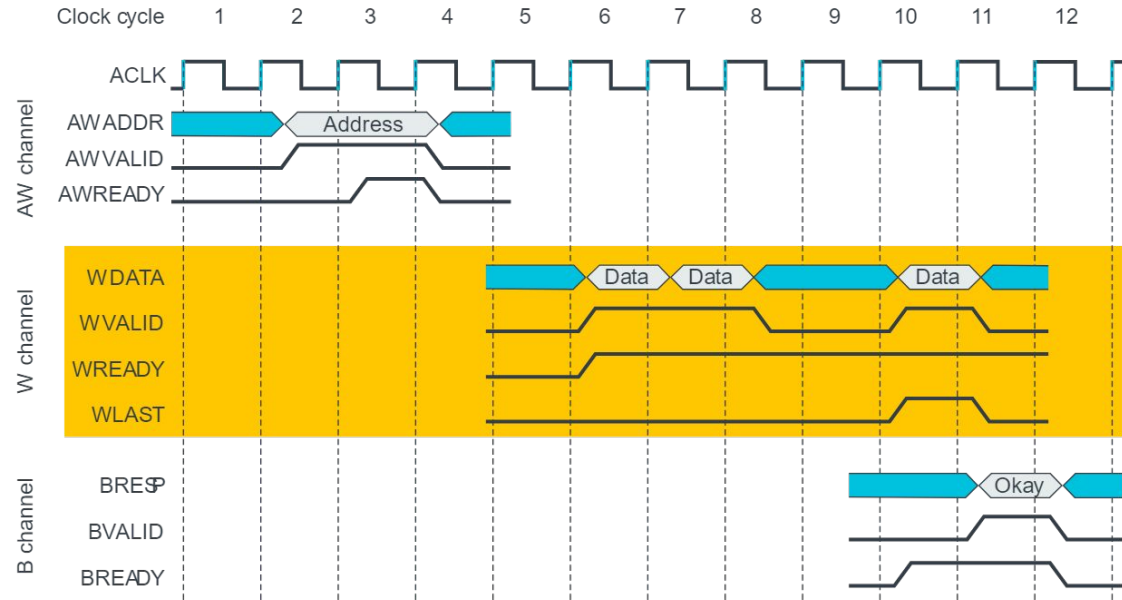
- All channels are bidirectional!!
- The arrow implies data flow direction, but not explicit input/output

AXI – Write Single

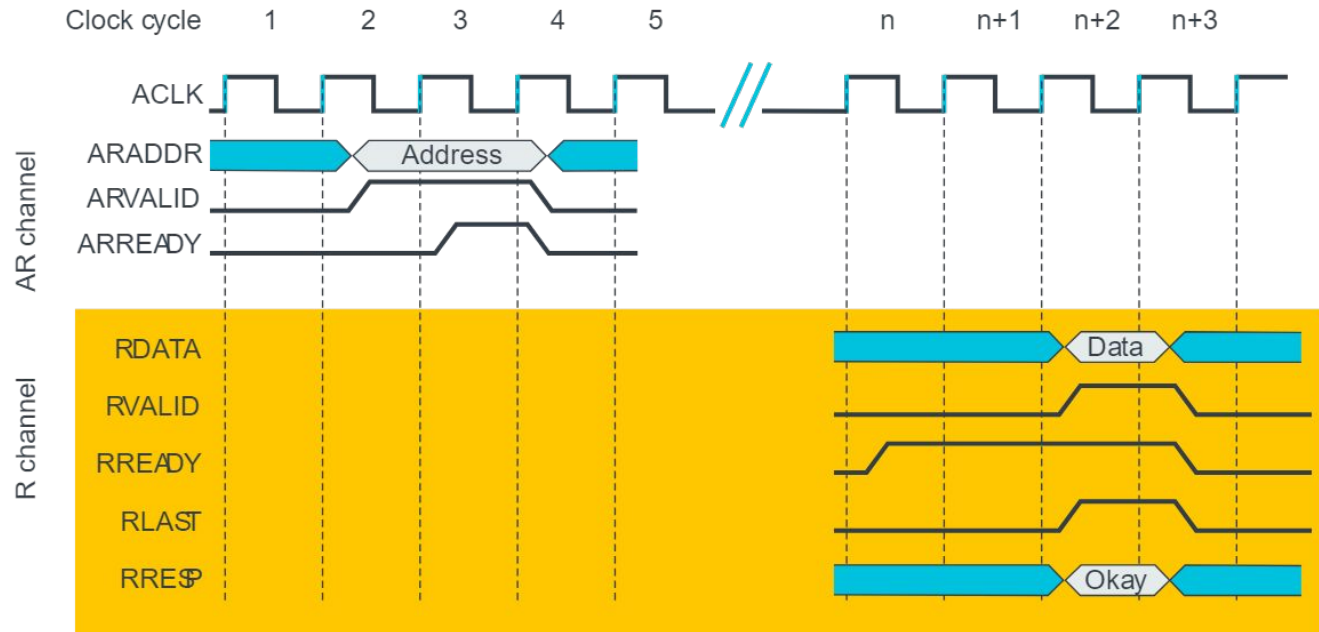
1. **AW (Address Write)**
Valid & Ready
starts of transaction
at **Address**
2. **W (Write)** **Valid & Ready**
3. **Su**
4. **Su**
5. **C**



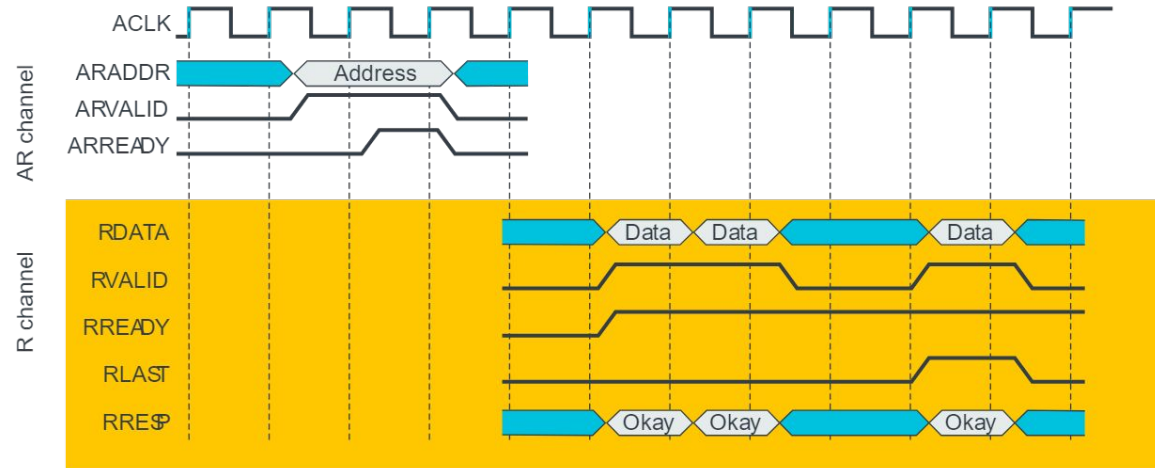
AXI – Write Multiple



AXI – Read Single



AXI – Read Multiple



References

- https://semiengineering.com/knowledge_centers/communications-io/on-chip-communications/network-on-chip-noc/
- <https://www.edn.com/why-network-on-chip-has-displaced-crossbar-switches-at-scale/>
- <https://ieeexplore.ieee.org/document/6966893>
- <https://books.google.com/books?id=uR3Vw9mYtpIC>
 - ISBN: 0080558283
- <https://developer.arm.com/documentation/102202/0300/Channel-transfers-and-transactions>
- <https://zipcpu.com/zipcpu/2017/11/07/wb-formal.html>
- <https://zipcpu.com/blog/2021/08/14/axiperf.html>
- <https://www.youtube.com/watch?v=Xl43BrQ8Z6c>